



IA02 - Rapport de projet

Vieillessement du système cardio-vasculaire – Etude de l'activité des peptides d'élastine et analyse data biodégradation Artères

**Molka GHADDAB, Zhaoyang Chen , Gaudeline
SIMO KAMWA , Pierre-Emeric Bocard, Belvine
Yemdjo Youmbi**



utt
UNIVERSITÉ DE TECHNOLOGIE
TROYES

INTRODUCTION

Contexte du Projet: L'élastine est un constituant important du corps se trouvant entre les cellules. Dans les parois vasculaires, sa présence est cruciale pour permettre l'absorption de la pression due aux battements cardiaques. Il est supposé que la dégradation de l'élastine, en formant ainsi des peptides, peut être un marqueur chimique activant des mécanismes de réparation.

Objectifs: Dans le présent cas, 12 trajectoires de peptides d'élastine nous sont données : 6 donnent lieu à une activité biologique, mais 6 ne semblent pas. Notre but est alors de pouvoir identifier une "conformation spatiale" des atomes de ces molécules qui serait la clé de génération de l'activité biologique, et d'ensuite pouvoir créer un générateur de peptides actives ou inactives.

Importance et Impact: Connaître le taux d'élastine active est un moyen efficace de mesurer un certain vieillissement du corps, car un corps abîmé a besoin de plus de réparation. L'apport de l'IA dans ce contexte servirait à accélérer grandement les processus d'analyses de quantité de données importantes.

EXTRACTION ET PREPARATION DES DONNEES

Fichiers : dataset.py puis reducing.py

Sources des Données: Notre source de données est le résultat de simulations réalisées par un doctorant avec le logiciel Gromacs et le champ de force OPLS-AA dans l'ensemble NPT (nombre de particules, pression et température constants ; pression = 1 atm, température = 300K), dans une boîte de 4x4x4 nm (donc positions entre 0 et 4 nm), avec un pas d'intégration de 2 fs (utilisation de l'algorithme SHAKE qui fixe la longueur des liaisons dans lesquelles un atome d'hydrogène est impliqué). Les conformations initiales des peptides sont en chaînes allongées (angles dièdres phi, psi = 180°, sauf pour les résidus proline phi = -75°, psi = 180°) et les extrémités ont été neutralisées.

Format des Données d'Entrée (.gro): Les fichiers avec l'extension gro contiennent une structure moléculaire au format Gromos87. Les fichiers gro peuvent être utilisés comme trajectoire en concaténant simplement les fichiers. Une tentative sera faite pour lire une valeur de temps à partir de la chaîne de titre dans chaque cadre, qui devrait être précédée de 't=', comme dans l'exemple ci-dessous :

MD of 2 waters, t= 0.0

6

```

1WATER OW1 1 0.126 1.624 1.679
1WATER HW2 2 0.190 1.661 1.747
1WATER HW3 3 0.177 1.568 1.613
2WATER OW1 4 1.275 0.053 0.622
2WATER HW2 5 1.337 0.002 0.680
2WATER HW3 6 1.326 0.120 0.568
1.82060 1.82060 1.82060

```

Les lignes contiennent les informations suivantes (de haut en bas) :

- chaîne de caractère de titre (temps en ps après 't=')
- nombre d'atomes
- une ligne pour chaque atome
- vecteurs de boîte (qui définit sa taille), valeurs : v1(x) v2(y) v3(z).

Ce format est fixe, c'est-à-dire que toutes les colonnes sont à une position fixe. Les colonnes contiennent les informations suivantes (de gauche à droite) :

- index du résidu
- nom du résidu
- nom de l'atome
- index de l'atome
- position (en nm, x y z en 3 colonnes avec 3 décimales)

Conversion des Données (.csv): À la base, il était prévu d'avoir un fichier csv pour toutes les peptides, mais compte tenu du poids total de ce fichier, cela faisait planter la RAM. Nous avons donc décidé de créer un fichier csv par peptide, que nous chargeons alors l'un après l'autre, après traitement du précédent. Ces fichiers csv possèdent les colonnes suivantes :

- 'Peptide' (le nom)
- 'Time' (le temps associé à la position)
- 'Residue_index' (index du résidu du fichier gro)
- 'Residue' (le nom)
- 'Atom' (le nom)
- 'Atom_index' (index de l'atome du fichier gro)
- 'x'
- 'y'
- 'z'

Analyse Exploratoire et Nettoyage des Données: Nous avons 12 peptides formées de chacune 6 acides aminés et dans de l'eau (modèle TIP3P, donc -H2O pour former les formules brutes) du temps $t = 0.0$ ps à $t = 200000$ ps, avec un pas de 5 ps, soit 40001 observations par peptide (et donc un total de 38 920 000 positions d'atomes) :

- APGVGV (inactif) (79 atomes) : ACE (résidu C₂H₃O) / ALA (Alanine C₃H₅NO) / PRO (proline C₅H₇NO) / GLY (Glycine C₂H₃NO) / VAL (Valine C₅H₉NO) / GLY / VAL / (NH₂)
- EGFEPG (actif) (87 atomes) : ACE / GLU (Acide glutamique C₅H₇NO₃) / GLY / PHE (Phénylalanine C₉H₉NO) / GLU / PRO / GLY / (NH₂)
- GVAPGV (actif) (79 atomes) : ACE / GLY / VAL / ALA / PRO / GLY / VAL / (NH₂)
- GVGVP (inactif) (79 atomes) : ACE / GLY / VAL / GLY / VAL / ALA / PRO / (NH₂)
- LGTIPG (actif) (89 atomes) : ACE / LEU (Leucine C₆H₁₁NO) / GLY / THR (Thréonine C₄H₇NO₂) / ILE (Isoleucine C₆H₁₁NO) / PRO / GLY / (NH₂)
- PGAIPG (actif) (80 atomes) : ACE / PRO / GLY / ALA / ILE / PRO / GLY / (NH₂)
- PGAYPG (actif) (82 atomes) : ACE / PRO / GLY / ALA / TYR (Tyrosine C₉H₉NO₂) / PRO / GLY / (NH₂)
- PGVGVA (inactif) (79 atomes) : ACE / PRO / GLY / VAL / GLY / VAL / ALA / (NH₂)
- VAPGVG (inactif) (79 atomes) : ACE / VAL / ALA / PRO / GLY / VAL / GLY / (NH₂)
- VGLAPG (actif) (82 atomes) : ACE / VAL / GLY / LEU / ALA / PRO / GLY / (NH₂)
- VGVAPG (actif) (79 atomes) : ACE / VAL / GLY / VAL / ALA / PRO / GLY / (NH₂)
- VVGPGA (inactif) (79 atomes) : ACE / VAL / VAL / GLY / PRO / GLY / ALA / (NH₂)

L'analyse exploratoire des 12 peptides nous a révélé qu'il n'y avait ni valeurs manquantes, ni doublons. Cependant, nous avons trouvé que les amplitudes de positions des atomes des peptides VGLAPG et VVGPGA, allant respectivement de -2 à 6 et de -0.2 à 4.2, étaient incohérentes, sortant des limites de la boîte. Mais compte tenu que cela concerne un nombre conséquent de positions (respectivement 1 000 000 et 150 000), et qu'en plus cela ne semble pas avoir d'impact sur les mouvements des peptides (comme si la boîte n'avait aucun effet sur le mouvement), nous avons décidé de ne pas supprimer ou remplacer ces mesures.

Réduction de la taille des données : Nous venons de voir que nous avons une quantité considérable de données. Cela nous a posé problème sur deux phases précises du projet. Le clustering, que nous allons traiter juste après, et la génération de données, que nous verrons en dernier. Pour le premier, sans réduction, le temps de calcul était d'environ 10h par peptide, et pour le second il devait être bien plus élevé encore.

Notre première solution était d'utiliser un GPU. Pour cela, nous avons regroupé dans un seul fichier l'entièreté du code du projet, que nous avons adapté au GPU. Pour le tester, nous avons utilisé un GPU externe RTX 3060. Mais cela s'est révélé être infructueux, multipliant au contraire par 4 les temps de calculs. On a donc fait le choix de réduire la taille des données pour obtenir au moins quelques résultats, tout en sachant que cela rendrait notre travail bien moins fiable.

Pour cela, nous avons utilisé deux réductions différentes, avec à chaque fois pour but de réduire par 10 le nombre de conformations pour en obtenir un total de 4001 par peptides. La première consiste à retirer 9 conformations sur 10 de manière régulière, nous permettant de conserver une durée d'expérience de 200ns au détriment de la précision des mouvements. La seconde au contraire consiste à ne garder que les 4001 premières conformations, nous permettant de conserver l'entièreté des mouvements sur une période donnée seulement.

CLUSTERING DES CONFORMATIONS PEPTIDIQUES

Fichier : clustering.py

Définition d'une distance : Notre objectif est d'à partir des conformations spatiales pouvoir déterminer si une peptide est active ou non. Il nous faut donc pouvoir déterminer si deux conformations sont identiques. Cependant, nous ne pouvons pas juste comparer des valeurs, puisque premièrement, il faut que l'on puisse considérer deux conformations identiques même si elles possèdent quelques différences. Un intervalle epsilon. Deuxièmement, il faut que l'on puisse tenir compte des translations et rotations. Il nous faut donc définir une distance. Nous en avons retenu deux :

- Soustraction du maximum et minimum des distances euclidiennes entre atomes identiques de 2 conformations d'une peptide. Cette soustraction simple doit permettre de prévenir en partie la translation et rotation. Mais elle est bernaible dans certaines conditions. C'est pourquoi nous avons aussi retenu une distance qui semble plus juste mais plus complexe en calcul (environ 3 fois plus long).

```
def distance_conformation(conf1, conf2):  
    """conf1 & 2 have to be numpy tab[[x1, y1, z1], [x2, y2, z2], ...] where all sub tab are one atom from one observation  
    (like t=0). Conf1 & 2 have to be from the same peptide.  
    Calcul the difference between max and min distance between same atoms from the both conformations  
    """  
    d = np.sqrt(np.sum((conf1 - conf2)**2, axis=-1))  
    return np.max(d) - np.min(d)
```

- On calcule la distance euclidienne entre un premier atome et tous les autres pour la 1ère et 2ème conformation, puis on calcule la différence entre ces 2 listes de distances pour voir si certains atomes ont bougé. Mais nous devons faire cela avec un deuxième atome qui n'est pas en mouvement avec le premier atome car dans ce cas, si un atome bouge en gardant la même distance avec le premier atome, il ne peut pas garder la même distance avec le deuxième atome. Donc on fait la même chose avec un faux atome (qui est juste une translation du premier atome de +1 nm pour être sûr qu'ils ne bougent pas entre eux), puis on soustrait les deux, et finalement, on prend la valeur absolue puis le maximum. C'est au final la distance que nous avons choisi d'utiliser.

```
def distance_conformation2(conf1, conf2):
    """conf1 & 2 have to be numpy tab[[x1, y1, z1], [x2, y2, z2], ...] where all sub tab are one atom from one observation
    (like t=0). Conf1 & 2 have to be from the same peptide.
    Calcul the euclidian distance between atom1 and all other atoms for conf1 and conf2, then he calcul the difference
    between these 2 list of distances to see if some atoms have moved.
    But we need to do that with a second atom which is not in mouv with the first atom because in this case, if one atom
    mouv while keeping the same distance with the first atom, he cont keep the same distance with the second atom.
    So we do the same thing with a false atom (which is just a translation of the first atom to be sure that they are not
    mouving between them)
    and finally, we take the absolute value and then the maximum|
    """
    return np.max(np.abs((np.linalg.norm(conf1 - conf1[0], axis=1) - np.linalg.norm(conf2 - conf2[0], axis=1)) -
                        (np.linalg.norm(conf1 - (conf1[0] + 1), axis=1) - np.linalg.norm(conf2 - (conf2[0] + 1), axis=1))))
```

Calcul de la Matrice des Distances: à l'aide de la distance choisie précédemment, on calcul la moitié de la matrice des distances classiquement (puisque l'autre moitié est identique, étant une matrice symétrique). Pour optimiser le temps de calcul, nous n'avons dans le code pas utilisé la fonction de distance, calculant la distance directement sur place (divisant grandement le temps de calcul). On a ainsi une matrice d'environ 12Go par peptide si pas de réduction (120Mo dans le cas contraire). Nous avons hésité à ne pas calculer la distance entre deux conformations à partir d'un certain écart de temps pour optimiser le poids et temps de calcul, mais nous nous sommes dit qu'il était tout à fait possible que la peptide retourne dans une conformation antérieure, et que donc nous ne devions pas considérer cela comme deux conformations différentes.

Algorithme DBSCAN: À partir de ces matrices des distances, nous pouvons maintenant réaliser un clustering. Notre choix s'est tout de suite porté sur le DBSCAN, ayant l'avantage de ne pas avoir besoin de déterminer à l'avance le nombre de clusters (donc de conformations différentes) et également de pouvoir déterminer des outliers automatiquement à partir d'un epsilon rentré en argument. Car effectivement, une peptide pourrait très bien se retrouver dans un état transitoire entre deux états stables.

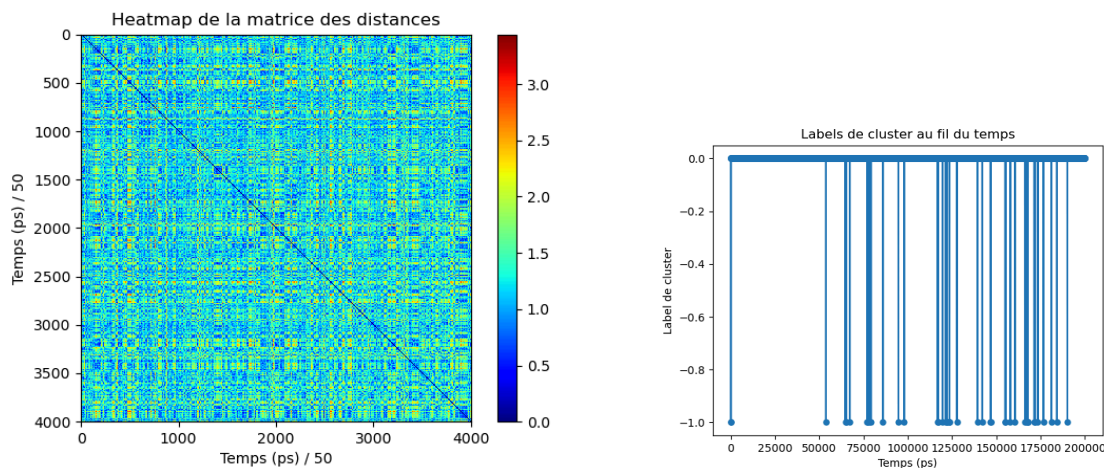
Le fonctionnement du DBSCAN est simple, puisque pour chaque conformation, il compte dans la matrice des distances le nombre de conformations à une distance inférieure à epsilon, et si ce nombre est supérieur à un certain seuil, alors ils sont catégorisés dans le même groupe.

Sélection des Hyperparamètres: Pour déterminer le meilleur modèle DBSCAN par rapport à nos données, on ne peut pas utiliser GridSearchCV, puisque ce dernier ne fonctionne que pour des modèles de classification supervisée. On fait donc la même chose que GridSearchCV mais à la main pour pouvoir l'adapter à un modèle de classification non-supervisée afin de faire varier epsilon et le seuil, en utilisant comme métrique d'évaluation la silhouette. Ainsi, epsilon varie de 0.2 à 1.0, et le seuil de 5 à 125.

Résultats du Clustering: Pour la plupart des peptides, les graphs sont assez similaires. Prenons EGFEPC (active) :

- 1ère réduction :

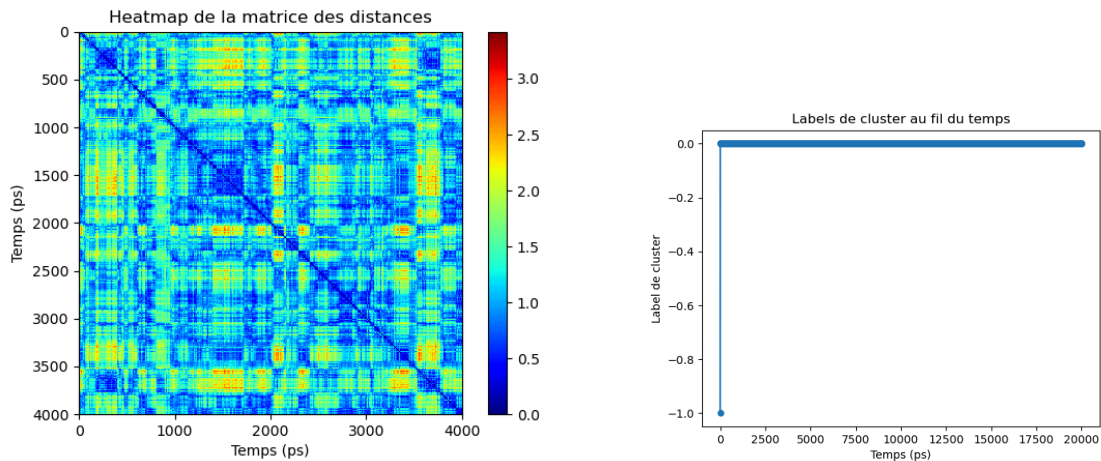
Modèle DBSCAN retenu : $\epsilon = 0.5$, $\text{min_samples} = 5$, silhouette = 0.544 (à savoir que 6 autres modèles DBSCAN ont obtenu exactement la même silhouette). Sachant que la silhouette varie de -1 à 1, ce n'est pas un mauvais score mais pas incroyable non plus.



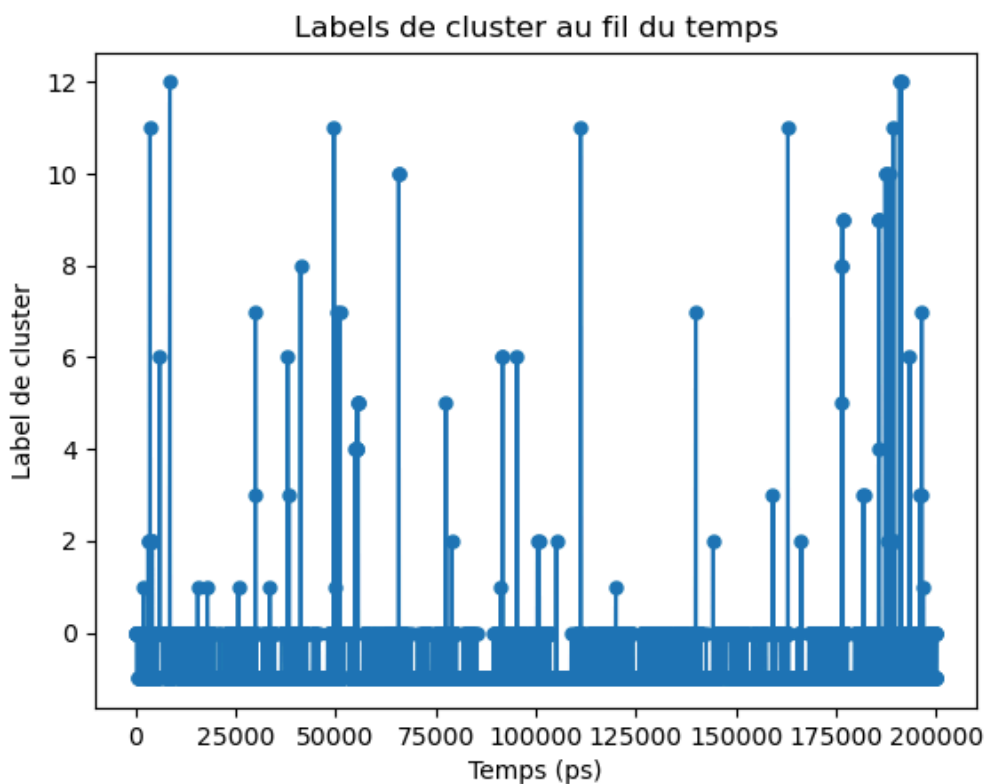
La première chose que l'on remarque sur la heatmap est que des conformations peuvent se retrouver très proches d'autres conformations pourtant très éloignées dans le temps, comme nous en avons fait l'hypothèse. On remarque également de petits carrés, signes d'états stables. Sur le graphique des labels, on remarque que le modèle n'a détecté qu'un seul cluster, ainsi que quelques outliers. Difficile de dire si ce sont des résultats cohérents, même si nous aurions tendance à dire que c'est plus un problème de modèle. Il serait intéressant de croiser avec d'autres modèles de clustering, comme le clustering hiérarchique.

- 2ème réduction (temps en ps / 5) :

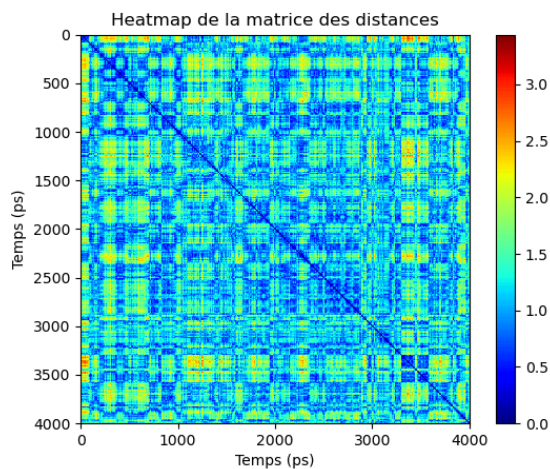
Modèle DBSCAN retenu : $\epsilon = 0.8$, $\text{min_samples} = 125$, silhouette = 0.59, soit un modèle bien différent du précédent pour une silhouette presque équivalente.



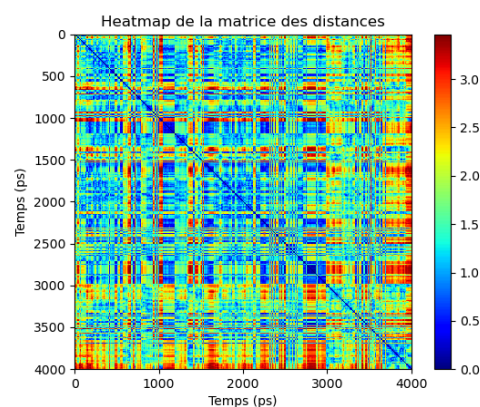
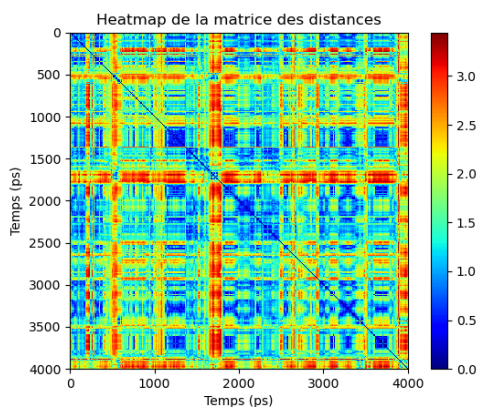
On remarque bien mieux les carrés sur la heatmap, on remarque aussi par exemple qu'on devrait avoir au moins 2 clusters, puisque les conformations entre 0 et 500 forment clairement un cluster différent que celles entre 1000 et 2000, étant assez éloignées de ces dernières. Pourtant, le graph des labels qui est cohérent avec le précédent, nous dit toujours qu'il n'y a qu'un seul cluster. Cela confirme donc que le modèle DBSCAN ne fonctionne pas comme on le voudrait. Et pratiquement tous les graphs de labels ne montrent qu'un seul cluster. Il y a une exception pour le graphs de VVGPGA (inactive) en 1ère réduction, qui parvient à trouver 13 clusters (avec tout de même une très grande majorité appartenant au 1er cluster :



Continuons seulement avec la 2nd réduction, qui semble plus pertinente. Si l'on prend CVGVAP qui est inactive on obtient quelque chose qui semble similaire à EGFEPC qui elle est active :



Pourtant, regardons VVGPGA (inactive) et VGLAPG (active) :



On a ici quelque chose qui semble complètement différent, avec beaucoup d'outliers. Compliqué de voir donc quelconque corrélation entre l'activité et les conformations pour le moment. Sachant qu'également, notre définition de distance peut être la cause du problème. Mais regardons ce que les modèles de classification supervisés pourront en tirer.

Classification des différentes peptides

Fichiers : preprocessing.py puis classification.py

Construction d'un nouveau csv : À partir du clustering, nous allons récupérer dans un nouveau csv unique et pour chaque peptides les informations que nous considérons possiblement utiles à la classification :

- 'Peptide' (le nom)
- 'Nb_conformations' (nombre de conformations uniques)
- 'Nb_outliers' (nombre de conformations hors clusters)
- 'Average_nb_state_per_conf' (nombre moyen de conformations identiques par état stable)
- 'Actif' (booléen)

Présentation des différents algorithmes de classification et de leur optimisation : Nous avons décidé de comparer 4 algorithmes de classification différents : Knn, RandomForest, Régression Logistique et SVM. Pour ces quatres algorithmes, nous effectuons une optimisation des hyperparamètres à l'aide d'un GridSearchCV et avec comme critère de notation l'accuracy. Puis on détermine le meilleur des 4.

Comparaisons des résultats de chaque algorithme :

- 1ère réduction :

```
Accuracy for knn: 0.3333333333333333
Accuracy for logreg: 0.6666666666666666
Accuracy for rf: 0.3333333333333333
Accuracy for svm: 0.6666666666666666
Best model is logreg with accuracy 0.6666666666666666
```

```
Best parameters for svm: {'svm__C': 0.001, 'svm__kernel': 'linear'}
```

```
Best parameters for logreg: {'logreg__C': 0.001, 'logreg__solver': 'lbfgs'}
```

- 2ème réduction :

```
Accuracy for knn: 0.3333333333333333
Accuracy for logreg: 0.6666666666666666
Accuracy for rf: 0.3333333333333333
Accuracy for svm: 0.6666666666666666
Best model is logreg with accuracy 0.6666666666666666
```

```
Best parameters for svm: {'svm__C': 0.001, 'svm__kernel': 'linear'}
```

```
Best parameters for logreg: {'logreg__C': 0.001, 'logreg__solver': 'lbfgs'}
```

On obtient donc les mêmes meilleurs modèles. Cependant, 0.66 reste un score assez mauvais. Mais cela n'est pas surprenant par le fait que premièrement, on a un échantillon de seulement 12 peptides, et que deuxièmement, notre clustering est très mauvais.

Modélisation d'un GAN pour la génération de peptides

Fichier : gru-GAN_3.0.py

Recherche d'une structure : Depuis le départ, nous savions que nous voulions faire un générateur conditionnel, pour que l'on puisse choisir si l'on désire produire une peptide active ou non. Puis, notre première idée a été d'utiliser un DCGAN pour faire intervenir la convolution. Pour chaque peptide, nous nous retrouvions avec une "image" de 3 pixel de large et 4001 pixel de long (pour pouvoir utiliser nos peptides réduites et donc raccourcir les temps de calculs), ainsi que d'un nombre de channels égal au nombre d'atome dans la peptide (comme les 3 channels d'une image couleur), car ils nous semblaient que réaliser une convolution 3d n'aurait de sens que si nous avions une représentation 3d de la peptide au cours du temps, mais ce n'est pas le cas. Mais nous nous sommes vite heurté au problème que chaque peptide n'a pas le même nombre d'atomes, et donc que l'on ne peut avoir un réseau neuronal adapté pour toutes les peptides. Une solution semblait possible en intégrant des channels vides pour arriver à égaliser le nombre d'atomes pour chaque peptide, mais cela s'est révélé être un non sens puisque cela signifiait créer plusieurs atomes immobiles et aux mêmes endroits, ce qui transformait complètement nos données.

Nous avons donc opté pour l'utilisation de neurones récurrents, et plus particulièrement de réseaux GRUs (Gated Recurrent Units), adapté à l'analyse temporelle puisque simulant une mémoire.

Architecture du Générateur: Le générateur est conçu pour générer les coordonnées 3D des atomes d'une peptide en se basant sur un bruit aléatoire et une étiquette de condition indiquant l'activité (active ou inactive). Il utilise un réseau GRU par atome, ce qui permet de capturer la dépendance temporelle et la structure séquentielle des coordonnées des atomes dans la peptide. Chaque GRU prend en entrée les coordonnées précédentes et l'étiquette conditionnelle, et génère les nouvelles coordonnées de l'atome correspondant.

Architecture du Discriminateur: Le discriminateur est conçu pour déterminer si une peptide est générée par le générateur ou provient des données réelles, ainsi que pour évaluer si la peptide est active ou non. Il utilise également un réseau GRU par atome, prenant en entrée les coordonnées des atomes et l'étiquette conditionnelle. Le discriminateur produit deux sorties : une pour la classification réelle/factice et une pour la classification active/inactive.

Ensemble de Données d'Entraînement: Les données utilisées pour l'entraînement sont constituées des positions atomiques des peptides à différents pas de temps. Chaque peptide est étiquetée comme active ou inactive en fonction de sa séquence. Pour préparer les données, nous avons utilisé un DataLoader personnalisé qui charge les fichiers CSV contenant les coordonnées atomiques et génère les étiquettes conditionnelles basées sur la séquence de la peptide.

Processus d'Entraînement: Le processus d'entraînement implique deux étapes principales pour chaque itération : la mise à jour du discriminateur et la mise à jour du générateur. Le discriminateur est entraîné à distinguer les peptides réelles des peptides générées et à évaluer leur activité. Le générateur est entraîné à produire des peptides qui non seulement semblent réelles mais qui sont aussi actives. Nous examinons également les pertes (losses) du générateur et du discriminateur au cours de l'entraînement pour s'assurer de la convergence et de la stabilité du modèle.

Évaluation du Modèle après entraînement : Pour évaluer le modèle, nous avons principalement observé la capacité du générateur à produire 5 peptides actives et 5 inactives, que nous sauvegardons sous forme de dataset. Puis nous les passons dans le discriminateur pour déterminer ce qu'il en pense. Dans un second temps, on les passe par le même processus que les véritables peptides, c'est-à-dire par le clustering puis par la classification supervisée au travers du meilleurs modèle déterminé plutôt.

Résultats et Analyse: Notre GAN s'est heurté à un problème : notre RAM. L'entraînement, qui semble assez long puisque nous n'avons jamais terminé une seule époque, surcharge notre RAM. Ainsi, nous n'avons pas de modèle fonctionnel. Il est probable que cela soit dû d'une part à la quantité de données, même une fois réduites, et d'autre part par le fait d'avoir un GRU par atome. Sachant que nous avons au maximum 89 atomes, cela représente une quantité importante d'hyperparamètres à optimiser. Mais il nous semblait qu'avoir un réseau GRU pour une peptide ne serait pas efficace. Avec du recul, nous aurions dû essayer de créer un tel modèle, plus simple donc à optimiser.

DISCUSSION ET CONCLUSION

Résumé des Résultats: Pour ce qui est de toute la partie concernant la détection, nous pouvons dire que les modèles de DBSCAN obtenus sont peu convaincant, et nécessitent soit des ajustements en testant d'autres valeurs possibles d'hyperparamètres, soit en changeant carrément de modèle pour par exemple des

modèles de clustering hiérarchique. Tout ceci a nécessairement eu un impact sur la classification supervisée qui en a suivi, et qui a ainsi donc également eu des résultats peu convaincants. Pour ce qui est du générateur, notre programme n'a jamais pu s'exécuter complètement dû à nos limitations techniques et sûrement aussi par notre choix de structure neuronal se composant de 89 GRU.

Limites du Projet: Les limites concernent aussi bien la détection que la génération. Premièrement, on a un manque de peptide pour pouvoir un échantillon important et améliorer la fiabilité des modèles. Deuxièmement, dû à la grande quantité de positions pour chaque peptide, on a un manque de capacité calculatoire avec nos moyens actuels, ce qui a également un impact important sur nos résultats, ayant dû avoir recours à une réduction des données.

RÉFÉRENCES

Articles Scientifiques:

- Thèse UTT sur ce jeu de donnée :
<https://theses.hal.science/tel-03219356/document>

Sites Web:

- Acides aminés :
<https://www.sigmaaldrich.com/FR/fr/technical-documents/technical-article/protein-biology/protein-structural-analysis/amino-acid-reference-chart>
- Explication des fichiers .gro :
<https://manual.gromacs.org/archive/5.0.3/online/gro.html>